Chapter 1

Considering the History and Uses of Data Science

IN THIS CHAPTER

- **Understanding data science history and uses**
- **Considering the flow of data in data science**
- **Working with various languages in data science**
- **Performing data science tasks quickly**

The burgeoning uses for data in the world today, along with the explosion of data sources, create a demand for people who have special skills to obtain, manage, and analyze information for the benefit of everyone. The data scientist develops and hones these special skills to perform such tasks on multiple levels, as described in the first two sections of this chapter.

Data needs to be funneled into acceptable forms that allow data scientists to perform their tasks. Even though the precise data flow varies, you can generalize it to a degree. The third section of the chapter gives you an overview of how data flow occurs.

As with anyone engaged in computer work today, a data scientist employs various programming languages to express the manipulation of

data in a repeatable manner. The languages that a data scientist uses, however, focus on outputs expected from given inputs, rather than on low-level control or a precise procedure, as a computer scientist would use. Because a data scientist may lack a formal programming education, the languages tend to focus on declarative strategies, with the data scientist expressing a desired outcome rather than devising a specific procedure. The fourth section of the chapter discusses various languages used by data scientists, with an emphasis on Python and R.

The final section of the chapter provides a very quick overview of getting tasks done quickly. Optimization without loss of precision is an incredibly difficult task and you see it covered a number of times in this book, but this introduction is enough to get you started. The overall goal of this first chapter is to describe data science and explain how a data scientist uses algorithms, statistics, data extraction, data manipulation, and a slew of other technologies to employ it as part of an analysis.

REMEMBER You don't have to type the source code for this chapter manually (or, actually at all, given that you use it only to obtain an understanding of the data flow process). In fact, using the downloadable source is a lot easier. The source code for this chapter appears in the `DSPD_0101_Quick_Overview.ipynb` source code file for Python. See the Introduction for details on how to find these source files.

Considering the Elements of Data Science

At one point, the world viewed anyone working with statistics as a sort of accountant or perhaps a mad scientist. Many people consider statistics and the analysis of data boring. However, data science is one of those occupations in which the more you learn, the more you want to learn. Answering one question often spawns more questions that are even more interesting than the one you just answered. However, what

makes data science so sexy is that you see it everywhere, used in an almost infinite number of ways. The following sections give you more details on why data science is such an amazing field of study.

Considering the emergence of data science

Data science is a relatively new term. William S. Cleveland coined the term in 2001 as part of a paper entitled "Data Science: An Action Plan for Expanding the Technical Areas of the Field of Statistics." It wasn't until a year later that the International Council for Science actually recognized data science and created a committee for it. Columbia University got into the act in 2003 by beginning publication of the *Journal of Data Science.*

REMEMBER However, the mathematical basis behind data science is centuries old because data science is essentially a method of viewing and analyzing statistics and probability. The first essential use of statistics as a term comes in 1749, but statistics are certainly much older than that. People have used statistics to recognize patterns for thousands of years. For example, the historian Thucydides (in his History of the Peloponnesian War) describes how the Athenians calculated the height of the wall of Platea in fifth century BC by counting bricks in an unplastered section of the wall. Because the count needed to be accurate, the Athenians took the average of the count by several solders.

The process of quantifying and understanding statistics is relatively new, but the science itself is quite old. An early attempt to begin documenting the importance of statistics appears in the ninth century, when Al-Kindi wrote *Manuscript on Deciphering Cryptographic Messages.* In this paper, Al-Kindi describes how to use a combination of statistics and frequency analysis to decipher encrypted messages. Even in the beginning, statistics saw use in the practical application of science for tasks that seemed virtually impossible to complete. Data

science continues this process, and to some people it might actually seem like magic.

Outlining the core competencies of a data scientist

As is true of anyone performing most complex trades today, the data scientist requires knowledge of a broad range of skills to perform the required tasks. In fact, so many different skills are required that data scientists often work in teams. Someone who is good at gathering data might team up with an analyst and someone gifted in presenting information. Finding a single person who possesses all the required skills would be hard. With this in mind, the following list describes areas in which a data scientist can excel (with more competencies being better):

- **Data capture:** It doesn't matter what sort of math skills you have if you can't obtain data to analyze in the first place. The act of capturing data begins by managing a data source using database-management skills. However, raw data isn't particularly useful in many situations; you must also understand the data domain so that you can look at the data and begin formulating the sorts of questions to ask. Finally, you must have data-modeling skills so that you understand how the data is connected and whether the data is structured.
- **Analysis:** After you have data to work with and understand the complexities of that data, you can begin to perform an analysis on it. You perform some analysis using basic statistical tool skills, much like those that just about everyone learns in college. However, the use of specialized math tricks and algorithms can make patterns in the data more obvious or help you draw conclusions that you can't draw by reviewing the data alone.
- **Presentation:** Most people don't understand numbers well. They can't see the patterns that the data scientist sees. Providing a graphical presentation of these patterns is important to help others visualize what the numbers mean and how to apply them in a meaningful way. More important, the presentation must tell a specific story so that the impact of the data isn't lost.

Linking data science, big data, and AI

Interestingly enough, the act of moving data around so that someone can perform analysis on it is a specialty called Extract, Transform, and Load (ETL). The ETL specialist uses programming languages such as Python to extract the data from a number of sources. Corporations tend not to keep data in one easily accessed location, so finding the data required to perform analysis takes time. After the ETL specialist finds the data, a programming language or other tool transforms it into a common format for analysis purposes. The loading process takes many forms, but this book relies on Python to perform the task. In a large, real-world operation, you might find yourself using tools such as Informatica, MS SSIS, or Teradata to perform the task.

REMEMBER Data science isn't necessarily a means to an end; it may instead be a step along the way. As a data scientist works through various datasets and finds interesting facts, these facts may act as input for other sorts of analysis and AI applications. For example, consider that your shopping habits often suggest what books you might like or where you might like to go for a vacation. Shopping or other habits can also help others understand other, sometimes less benign, activities as well. *Machine Learning For Dummies* and *Artificial Intelligence For Dummies*, both by John Paul Mueller and Luca Massaron (Wiley), help you understand these other uses of data science. For now, consider the fact that what you learn in this book can have a definite effect on a career path that will go many other places.

Understanding the role of programming

A data scientist may need to know several programming languages in order to achieve specific goals. For example, you may need SQL knowledge to extract data from relational databases. Python can help you perform data loading, transformation, and analysis tasks. However, you might choose a product such as MATLAB (which has its own programming language) or PowerPoint (which relies on VBA) to present

the information to others. (If you're interested to see how MATLAB compares to the use of Python, you can get the book, *MATLAB For Dummies,* by John Paul Mueller [Wiley].) The immense datasets that data scientists rely on often require multiple levels of redundant processing to transform into useful processed data. Manually performing these tasks is time consuming and error prone, so programming presents the best method for achieving the goal of a coherent, usable data source.

Given the number of products that most data scientists use, sticking to just one programming language may not be possible. Yes, Python can load data, transform it, analyze it, and even present it to the end user, but the process works only when the language provides the required functionality. You may have to choose other languages to fill out your toolkit. The languages you choose depend on a number of criteria. Here are some criteria you should consider:

- How you intend to use data science in your code (you have a number of tasks to consider, such as data analysis, classification, and regression)
- Your familiarity with the language
- The need to interact with other languages
- The availability of tools to enhance the development environment
- The availability of APIs and libraries to make performing tasks easier

Defining the Role of Data in the World

This section of the chapter is too short. It can't even begin to describe the ways in which data will affect you in the future. Consider the following subsections as offering tantalizing tidbits —appetizers that can whet your appetite for exploring the world of data and data science further. The applications listed in these sections are already common in some settings. You probably used at least one of them today, and quite likely more than just one. After reading the following sections, you might want to take the time to consider all the ways in which data currently affects your life. The use of data to perform amazing feats is

really just the beginning. Humanity is at the cusp of an event that will rival the Industrial Revolution (see https://www.history.com/topics/industrial-revolution/industrial-revolution), and the use of data (and its associated technologies, such as AI, machine learning, and deep learning) is actually quite immature at this point.

Enticing people to buy products

Demographics, those vital or social statistics that group people by certain characteristics, have always been part art and part science. You can find any number of articles about getting your computer to generate demographics for clients (or potential clients). The use of demographics is wide ranging, but you see them used for things like predicting which product a particular group will buy (versus that of the competition). Demographics are an important means of categorizing people and then predicting some action on their part based on their group associations. Here are the methods that you often see cited for AIs when gathering demographics:

- **Historical:** Based on previous actions, an AI generalizes which actions you might perform in the future.
- **Current activity:** Based on the action you perform now and perhaps other characteristics, such as gender, a computer predicts your next action.
- **Characteristics:** Based on the properties that define you, such as gender, age, and area where you live, a computer predicts the choices you are likely to make.

WARNING You can find articles about AI's predictive capabilities that seem almost too good to be true. For example, the article at https://medium.com/@demografy/artificial-intelligence-can-now-predict-demographic-characteristics-knowing-only-your-name-6749436a6bd3 says that AI can now predict your demographics based solely on your name. The company in that article, Demografy

(`https://demografy.com/`), claims to provide gender, age, and cultural affinity based solely on name. Even though the site claims that it's 90 to 95 percent accurate (see the Is Demografy Accurate answer at `https://demografy.com/faq` for details), this statistic is unlikely because some names are gender ambiguous, such as Renee, and others are assigned to one gender in some countries and another gender in others. In fact, the answer on the Demografy site seems to acknowledge this issue by saying the outcome "heavily depends on your particular list and may show considerably different results than these averages". Yes, demographic prediction can work, but exercise care before believing everything that these sites tell you.

If you want to experiment with demographic prediction, you can find a number of APIs online. For example, the DeepAI API at `https://deepai.org/machine-learning-model/demographic-recognition` promises to help you predict age, gender, and cultural background based on a person's appearance in a video. Each of the online APIs do specialize, so you need to choose the API with an eye toward the kind of input data you can provide.

Keeping people safer

You already have a good idea of how data might affect you in ways that keep you safer. For example, statistics help car designers create new designs that provide greater safety for the occupant and sometimes other parties as well. Data also figures into calculations for things like

- Medications
- Medical procedures
- Safety equipment
- Safety procedures
- How long to keep the crosswalk signs lit

Safety goes much further, though. For example, people have been trying to predict natural disasters for as long as there have been people

and natural disasters. No one wants to be part of an earthquake, tornado, volcanic eruption, or any other natural disaster. Being able to get away quickly is the prime consideration in such cases, given that humans can't control their environment well enough yet to prevent any natural disaster.

Data managed by deep learning provides the means to look for extremely subtle patterns that boggle the minds of humans. These patterns can help predict a natural catastrophe, according to the article on Google's solution at `http://www.digitaljournal.com/tech-and-science/technology/google-to-use-ai-to-predict-natural-disasters/article/533026`. The fact that the software can predict any disaster at all is simply amazing. However, the article at `http://theconversation.com/ai-could-help-us-manage-natural-disasters-but-only-to-an-extent-90777` warns that relying on such software exclusively would be a mistake. Overreliance on technology is a constant theme throughout this book, so don't be surprised that deep learning is less than perfect in predicting natural catastrophes as well.

Creating new technologies

New technologies can cover a very wide range of applications. For example, you find new technologies for making factories safer and more efficient all the time. Space travel requires an inordinate number of new technologies. Just consider how the data collected in the past affects things like smart phone use and the manner in which you drive your car.

However, a new technology can take an interesting twist, and you should look for these applications as well. You probably have black-and-white videos or pictures of family members or special events that you'd love to see in color. Color consists of three elements: hue (the actual color); value (the darkness or lightness of the color); and saturation (the intensity of the color). You can read more about these ele-

ments at http://learn.leighcotnoir.com/artspeak/elements-color/hue-value-saturation/. Oddly enough, many artists are color-blind and make strong use of color value in their creations (read https://www-nytimes-com.ezproxy.sfpl.org/2017/12/23/books/a-colorblind-artist-illustrator-childrens-books.html as one of many examples). So having hue missing (the element that black-and-white art lacks) isn't the end of the world. Quite the contrary: Some artists view it as an advantage (see https://www.artsy.net/article/artsy-editorial-the-advantages-of-being-a-colorblind-artist for details).

When viewing something in black and white, you see value and saturation but not hue. *Colorization* is the process of adding the hue back in. Artists generally perform this process using a painstaking selection of individual colors, as described at https://fstoppers.com/video/how-amazing-colorization-black-and-white-photos-are-done-5384 and https://www.diyphotography.net/know-colors-add-colorizing-black-white-photos/. However, AI has automated this process using Convolutional Neural Networks (CNNs), as described at https://emerj.com/ai-future-outlook/ai-is-colorizing-and-beautifying-the-world/.

REMEMBER The easiest way to use CNN for colorization is to find a library to help you. The Algorithmia site at https://demos.algorithmia.com/colorize-photos/ offers such a library and shows some example code. You can also try the application by pasting a URL into the supplied field. The article at https://petapixel.com/2016/07/14/app-magically-turns-bw-photos-color-ones/ describes just how well this application works. It's absolutely amazing!

Performing analysis for research

Most people think that research focuses only on issues like health, consumerism, or improving efficiency. However, research takes a great many other forms as well, many of which you'll never even hear about, such as figuring out how people move in order to keep them safer. Think about a manikin for a moment. You can pose the manikin in various ways to see how that pose affects an environment, such as in car crash research. However, manikins are simply snapshots in a process that happens in real time. In order to see how people interact with their environment, you must pose the people in a fluid, real time, manner using a strategy called *person poses*.

Person poses don't tell you who is in a video stream, but rather what elements of a person are in the video stream. For example, using a person pose can tell you whether the person's elbow appears in the video and where it appears. The article at `https://medium.com/tensorflow/real-time-human-pose-estimation-in-the-browser-with-tensorflow-js-7dd0bc881cd5` tells you more about how this whole visualization technique works. In fact, you can see how the system works through a short animation of one person in the first case and three people in the second case.

Person poses can have all sorts of useful purposes. For example, you might use a person pose to help people improve their form for various kinds of sports — everything from golf to bowling. A person pose could also make new sorts of video games possible. Imagine being able to track a person's position for a game without the usual assortment of cumbersome gear. Theoretically, you could use person poses to perform crime-scene analysis or to determine the possibility of a person committing a crime.

Another interesting application of pose detection is for medical and rehabilitation purposes. Software powered by data managed by deep learning techniques could tell you whether you're doing your exercises correctly and track your improvements. An application of this sort could support the work of a professional rehabilitator by taking

care of you when you aren't in a medical facility (an activity called telerehabilitation; see https://matrc.org/telerehabilitation-telepractice for details).

REMEMBER Fortunately, you can at least start working with person poses today using the tfjs-models (PoseNet) library at https://github.com/tensorflow/tfjs-models/tree/master/posenet. You can see it in action with a webcam, complete with source code, at https://ml5js.org/docs/posenet-webcam. The example takes a while to load, so you need to be patient.

Providing art and entertainment

Book 5, Chapter 4 provides you with some good ideas on how deep learning can use the content of a real-world picture and an existing master painter (live or dead) for style to create a combination of the two. In fact, some pieces of art generated using this approach are commanding high prices on the auction block. You can find all sorts of articles on this particular kind of art generation, such as the *Wired* article at https://www.wired.com/story/we-made-artificial-intelligence-art-so-can-you/.

However, even though pictures are nice for hanging on the wall, you might want to produce other kinds of art. For example, you can create a 3-D version of your picture using products like Smoothie 3-D. The articles at https://styly.cc/tips/smoothie-3d/ and https://3dprint.com/38467/smoothie-3d-software/ describe how this software works. It's not the same as creating a sculpture; rather, you use a 3-D printer to build a 3-D version of your picture. The article at https://thenextweb.com/artificial-intelligence/2018/03/08/try-this-ai-experiment-that-converts-2d-images-to-3d/ offers an experiment that you can perform to see how the process works.

REMEMBER The output of an AI doesn't need to consist of something visual, either. For example, deep learning enables you to create music based on the content of a picture, as described at https://www.cnet.com/news/baidu-ai-creates-original-music-by-looking-at-pictures-china-google/ . This form of art makes the method used by AI clearer. The AI transforms content that it doesn't understand from one form to another. As humans, we see and understand the transformation, but all the computer sees are numbers to process using clever algorithms created by other humans.

Making life more interesting in other ways

Data is part of your life. You really can't perform too many activities anymore that don't have data attached to them in some way. For example, consider gardening. You might think that digging in the earth, planting seeds, watering, and harvesting fruit has nothing to do with data, yet the seeds you use likely rely on research conducted as the result of gathering data. The tools you use to dig are now ergonomically designed based on human research studies. The weather reports you use to determine whether to water or not rely on data. The clothes you wear, the shoes you employ to work safely, and even the manner in which you work are all influenced by data. Now, consider that gardening is a relatively nontechnical task that people have performed for thousands of years, and you get a good feel for just how much data affects your daily life.

Creating the Data Science Pipeline

Data science is partly art and partly engineering. Recognizing patterns in data, considering what questions to ask, and determining which algorithms work best are all part of the art side of data science.
However, to make the art part of data science realizable, the engineer-

ing part relies on a specific process to achieve specific goals. This process is the data science pipeline, which requires the data scientist to follow particular steps in the preparation, analysis, and presentation of the data. The following sections help you understand the data science pipeline better so that you can understand how the book employs it during the presentation of examples.

Preparing the data

The data that you access from various sources doesn't come in an easily packaged form, ready for analysis — quite the contrary. The raw data not only may vary substantially in format, but you may also need to transform it to make all the data sources cohesive and amenable to analysis. Transformation may require changing data types, the order in which data appears, and even the creation of data entries based on the information provided by existing entries.

Performing exploratory data analysis

The math behind data analysis relies on engineering principles in that the results are provable and consistent. However, data science provides access to a wealth of statistical methods and algorithms that help you discover patterns in the data. A single approach doesn't ordinarily do the trick. You typically use an iterative process to rework the data from a number of perspectives. The use of trial and error is part of the data science art.

Learning from data

As you iterate through various statistical analysis methods and apply algorithms to detect patterns, you begin learning from the data. The data might not tell the story that you originally thought it would, or it might have many stories to tell. Discovery is part of being a data scientist. In fact, it's the fun part of data science because you can't ever know in advance precisely what the data will reveal to you.

REMEMBER Of course, the imprecise nature of data and the finding of seemingly random patterns in it means keeping an open mind. If you have preconceived ideas of what the data contains, you won't find the information it actually does contain. You miss the discovery phase of the process, which translates into lost opportunities for both you and the people who depend on you.

Visualizing

Visualization means seeing the patterns in the data and then being able to react to those patterns. It also means being able to see when data is not part of the pattern. Think of yourself as a data sculptor — removing the data that lies outside the patterns (the outliers) so that others can see the masterpiece of information beneath. Yes, you can see the masterpiece, but until others can see it, too, it remains in your vision alone.

Obtaining insights and data products

The data scientist may seem to simply be looking for unique methods of viewing data. However, the process doesn't end until you have a clear understanding of what the data means. The insights you obtain from manipulating and analyzing the data help you to perform real-world tasks. For example, you can use the results of an analysis to make a business decision.

In some cases, the result of an analysis creates an automated response. For example, when a robot views a series of pixels obtained from a camera, the pixels that form an object have special meaning, and the robot's programming may dictate some sort of interaction with that object. However, until the data scientist builds an application that can load, analyze, and visualize the pixels from the camera, the robot doesn't see anything at all.

Comparing Different Languages Used for Data Science

None of the existing programming languages in the world can do everything. One such language endeavor, Ada, has received limited success because the language is incredibly difficult to learn (see `https://www.nap.edu/read/5463/chapter/3` and `https://news.ycombinator.com/item?id=7824570` for details). The problem is that if you make a language robust enough to do everything, it's too complex to do anything. Consequently, as a data scientist, you likely need exposure to a number of languages, each of which has a forte in a particular aspect of data science development. The following sections help you to better understand the languages used for data science, with a special emphasis on Python and R, the languages supported by this book.

Obtaining an overview of data science languages

Many different programming languages exist, and most were designed to perform tasks in a certain way or even make a particular profession's work easier to do. Choosing the correct tool makes your life easier. It's akin to using a hammer instead of a screwdriver to drive a screw. Yes, the hammer works, but the screwdriver is much easier to use and definitely does a better job. Data scientists usually use only a few languages because they make working with data easier. With this idea in mind, here are the top languages for data science work in order of preference:

- **Python (general purpose):** Many data scientists prefer to use Python because it provides a wealth of libraries, such as NumPy, SciPy, MatPlotLib, pandas, and Scikit-learn, to make data science tasks significantly easier. Python is also a precise language that makes using multiprocessing on large datasets easier, thereby reducing the time required to analyze them. The data science community has also stepped up with specialized IDEs, such as Anaconda, that implement the Jupyter Notebook concept, which makes working with data science calculations significantly easier. (Chapter 3 of this

minibook demonstrates how to use Jupyter Notebook, so don't worry about it in this chapter.) In addition to all these aspects in Python's favor, it's also an excellent language for creating *glue code* (code that is used to connect various existing code elements together into a cohesive whole) with languages such as C/C++ and Fortran. The Python documentation actually shows how to create the required extensions. Most Python users rely on the language to see patterns, such as allowing a robot to see a group of pixels as an object. It also sees use for all sorts of scientific tasks.

- **R (special purpose statistical):** In many respects, Python and R share the same sorts of functionality but implement it in different ways. Depending on which source you view, Python and R have about the same number of proponents, and some people use Python and R interchangeably (or sometimes in tandem). Unlike Python, R provides its own environment, so you don't need a third-party product such as Anaconda. However, Chapter 3 of this minibook shows how you can use R in Jupyter Notebook so that you can use a single IDE for all your needs. Unfortunately, R doesn't appear to mix with other languages with the ease that Python provides.
- **SQL (database management):** The most important thing to remember about Structured Query Language (SQL) is that it focuses on data rather than tasks. (This distinction makes it a full-fledged language for a data scientist, but only part of a solution for a computer scientist.) Businesses can't operate without good data management — the data is the business. Large organizations use some sort of relational database, which is normally accessible with SQL, to store their data. Most Database Management System (DBMS) products rely on SQL as their main language, and DBMS usually has a large number of data analysis and other data science features built in. Because you're accessing the data natively, you often experience a significant speed gain in performing data science tasks this way. Database Administrators (DBAs) generally use SQL to manage or manipulate the data rather than necessarily perform detailed analysis of it. However, the data scientist can also use SQL for various data science tasks and make the resulting scripts available to the DBAs for their needs.
- **Java (general purpose):** Some data scientists perform other kinds of programming that require a general-purpose, widely adapted, and popular language. In addition to providing access to a large number of libraries (most of which aren't actually all that useful for data science, but do work for other needs), Java supports object orientation better than any of the other languages in this list. In addition, it's strongly typed and tends to run quite quickly. Consequently, some people prefer it for finalized code. Java isn't a

good choice for experimentation or ad hoc queries. Oddly enough, an implementation of Java exists for Jupyter Notebook, but it isn't refined and is not usable for data science work at this time. (You can find helpful information about the Jupyter Java implementation at `https://blog.frankel.ch/teaching-java-jupyter-notebooks/`, `https://github.com/scijava/scijava-jupyter-kernel`, and `https://github.com/jupyter/jupyter/wiki/Jupyter-kernels`.)

- **Scala (general purpose):** Because Scala uses the Java Virtual Machine (JVM), it does have some of the advantages and disadvantages of Java. However, like Python, Scala provides strong support for the functional programming paradigm, which uses lambda calculus as its basis (see *Functional Programmming For Dummies*, by John Paul Mueller [Wiley] for details). In addition, Apache Spark is written in Scala, which means that you have good support for cluster computing when using this language. Think huge dataset support. Some of the pitfalls of using Scala are that it's hard to set up correctly, it has a steep learning curve, and it lacks a comprehensive set of data science–specific libraries.

Defining the pros and cons of using Python

Given the right data sources, analysis requirements, and presentation needs, you can use Python for every part of the data science pipeline. In fact, that's precisely what you do in this book. Every example uses Python to help you understand another part of the data science equation. Of all the languages you could choose for performing data science tasks, Python is the most flexible and capable because it supports so many third-party libraries devoted to the task. The following sections help you better understand why Python is such a good choice for many (if not most) data science needs.

Considering the shifting profile of data scientists

Some people view the data scientist as an unapproachable nerd who performs miracles on data with math. The data scientist is the person behind the curtain in an Oz-like experience. However, this perspective is changing. In many respects, the world now views the data scientist as either an adjunct to a developer or as a new type of developer. The

ascendance of applications of all sorts that can learn is the essence of this change. For an application to learn, it has to be able to manipulate large databases and discover new patterns in them. In addition, the application must be able to create new data based on the old data — making an informed prediction of sorts. The new kinds of applications affect people in ways that would have seemed like science fiction just a few years ago. Of course, the most noticeable of these applications define the behaviors of robots that will interact far more closely with people tomorrow than they do today.

From a business perspective, the necessity of fusing data science and application development is obvious: Businesses must perform various sorts of analysis on the huge databases they have collected — to make sense of the information and use it to predict the future. In truth, however, the far greater impact of the melding of these two branches of science — data science and application development — will be felt in terms of creating altogether new kinds of applications, some of which aren't even possible to imagine with clarity today. For example, new applications could help students learn with greater precision by analyzing their learning trends and creating new instructional methods that work for that particular student. This combination of sciences might also solve a host of medical problems that seem impossible to solve today — not only in keeping disease at bay, but also by solving problems, such as how to create truly usable prosthetic devices that look and act like the real thing.

Working with a multipurpose, simple, and efficient language

Many different ways are available for accomplishing data science tasks. This book covers only one of the myriad methods at your disposal. However, Python represents one of the few single-stop solutions that you can use to solve complex data science problems. Instead of having to use a number of tools to perform a task, you can simply use a single language, Python, to get the job done. The Python difference is the large number scientific and math libraries created for it by third

parties. Plugging in these libraries greatly extends Python and allows it to easily perform tasks that other languages could perform, but with great difficulty.

TIP Python's libraries are its main selling point; however, Python offers more than reusable code. The most important thing to consider with Python is that it supports four different coding styles:

- **Functional:** Treats every statement as a mathematical equation and avoids any form of state or mutable data. The main advantage of this approach is having no side effects to consider. In addition, this coding style lends itself better than the others to parallel processing because you have no state to consider. Many developers prefer this coding style for recursion and for lambda calculus.
- **Imperative:** Performs computations as a direct change to program state. This style is especially useful when manipulating data structures and produces elegant, but simple, code.
- **Object-oriented:** Relies on data fields that are treated as objects and manipulated only through prescribed methods. Python doesn't fully support this coding form because it can't implement features such as data hiding. However, this is a useful coding style for complex applications because it supports encapsulation and polymorphism. This coding style also favors code reuse.
- **Procedural:** Treats tasks as step-by-step iterations in which common tasks are placed in functions that are called as needed. This coding style favors iteration, sequencing, selection, and modularization.

Defining the pros and cons of using R

The standard download of R is a combination of an environment and a language. It's a form of the S programming language, which John Chambers originally created at Bell Laboratories to make working with statistics easier. Rick Becker and Allan Wilks eventually added to the S programming language as well. The goal of the R language is to turn ideas into software quickly and easily. In other words, R is a lan-

guage designed to help someone who doesn't have much programming experience create code without a huge learning curve.

This book uses R instead of S because R is a free, downloadable product that can run most S code without modification; in contrast, you have to pay for S. Given the examples used in the book, R is a great choice. You can read more about R in general at `https://www.r-project.org/about.html`.

WARNING You don't want to make sweeping generalizations about the languages used for data science because you must also consider how the languages are used within the field (such as performing machine learning or deep learning tasks). Both R and Python are popular languages for different reasons. Articles such as "In data science, the R language is swallowing Python" (`http://www.infoworld.com/article/2951779/application-development/in-data-science-the-r-language-is-swallowing-python.html`) initially seem to say that R is becoming more popular for some reason, which isn't clearly articulated. The author wisely backs away from this statement by pointing out that R is best used for statistical purposes and Python is a better general-purpose language. The best developers always have an assortment of programming tools in their tool belts to make performing tasks easier. Languages address developer needs, so you need to use the right language for the job. After all, all languages ultimately become machine code that a processor understands — an extremely low-level, processor-specific language that few developers understand any longer because high-level programming languages make development easier.

You can get a basic copy of R from the Comprehensive R Archive Network (CRAN) site at `https://cran.r-project.org/`. The site provides both source code versions and compiled versions of the R distribution for various platforms. Unless you plan to make your own

changes to the basic R support or want to delve into how R works, getting the compiled version is always better. If you use RStudio, you must also download and install a copy of R.

REMEMBER This book uses a version of R specially designed for use in Jupyter Notebook (as described in Chapter 3 of this minibook). Because you can work with Python and R using the same IDE, you save time and effort because now you don't have to learn a separate IDE for R. However, you might ultimately choose to work with a specialized R environment to obtain language help features that Jupyter Notebook doesn't provide. If you use a different IDE, the screenshots in the book won't match what you see onscreen, and the downloadable source code files may not load without error (but should still work with minor touchups).

The RStudio Desktop version (`https://www.rstudio.com/products/rstudio/#Desktop`) can make the task of working with R even easier. This product is a free download, and you can get it in Linux (Debian/Ubuntu, RedHat/CentOS, and SUSE Linux), Mac, and Windows versions. The book doesn't use the advanced features found in the paid version of the product, nor will you require the RStudio Server features for the examples.

You can try other R distributions if you find that you don't like Jupyter Notebook or RStudio. The most common alternative distributions are StatET (`http://www.walware.de/goto/statet`), Red-R (`https://decisionstats.com/2010/09/28/red-r-1-8-groovy-gui/` or `http://www.red-r.org/`), and Rattle (`http://rattle.togaware.com/`). All of them are good products, but RStudio appears to have the strongest following and is the simplest product to use outside Jupyter Notebook. You can read discussions about the various choices online at places such as

https://www.quora.com/What-are-the-best-choices-for-an-R-IDE.

Learning to Perform Data Science Tasks Fast

It's time to see the data science pipeline in action. Even though the following sections use Python to provide a brief overview of the process you explore in detail in the rest of the book, they also apply to using R. Throughout the book, you see Python used directly in the text for every example, with some R additions. The downloadable source contains R versions of the examples that reflect the capabilities that R provides.

You won't actually perform the tasks in the following sections. In fact, you don't find installation instructions for Python until Chapter 3, so in this chapter, you can just follow along in the text. This book uses a specific version of Python and an IDE called Jupyter Notebook, so please wait until Chapter 3 to install these features (or skip ahead, if you insist, and install them now). Don't worry about understanding every aspect of the process at this point. The purpose of these sections is to help you gain an understanding of the flow of using Python to perform data science tasks. Many of the details may seem difficult to understand at this point, but the rest of the book will help you understand them.

REMEMBER The examples in this book rely on a web-based application named Jupyter Notebook. The screenshots you see in this and other chapters reflect how Jupyter Notebook looks in Firefox on a Windows 7 system. The view you see will contain the same data, but the actual interface may differ a little depending on platform (such as using a notebook instead of a desktop system), operating system, and browser. Don't worry if you see some slight differences between your display and the screenshots in the book.

Loading data

Before you can do anything, you need to load some data. The book shows you all sorts of methods for performing this task. In this case, Figure 1-1 shows how to load a dataset called Boston that contains housing prices and other facts about houses in the Boston area. The code places the entire dataset in the `boston` variable and then places parts of that data in variables named `X` and `y`. Think of variables as you would storage boxes. The variables are important because they enable you to work with the data.

Loading data

```
In [1]: from sklearn.datasets import load_boston
        boston = load_boston()
        X, y = boston.data,boston.target
```

FIGURE 1-1: Loading data into variables so that you can manipulate it.

Training a model

Now that you have some data to work with, you can do something with it. All sorts of algorithms are built into Python. Figure 1-2 shows a linear regression model. Again, don't worry precisely how this works; later chapters discuss linear regression in detail. The important thing to note in Figure 1-2 is that Python lets you perform the linear regression using just two statements and to place the result in a variable named `hypothesis`.

Training a model

```
In [2]: from sklearn.linear_model import LinearRegression
        hypothesis = LinearRegression(normalize=True)
        hypothesis.fit(X,y)
Out[2]: LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None, normalize
        =True)
```

FIGURE 1-2: Using the variable content to train a linear regression model.

Viewing a result

Performing any sort of analysis doesn't pay unless you obtain some benefit from it in the form of a result. This book shows all sorts of ways to view output, but Figure 1-3 starts with something simple. In this case, you see the coefficient output from the linear regression analysis.

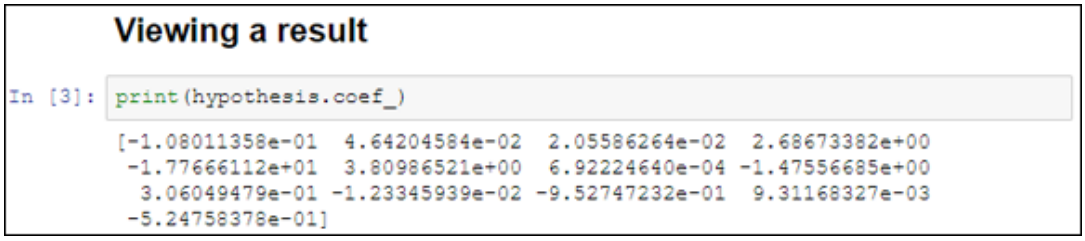

FIGURE 1-3: Outputting a result as a response to the model.

TIP One of the reasons that this book uses Jupyter Notebook is that the product helps you to create nicely formatted output as part of creating the application. Look again at Figure 1-3 and you see a report that you could simply print and offer to a colleague. The output isn't suitable for many people, but those experienced with Python and data science will find it quite usable and informative.

Table of contents collapsed